In case of a hypothesis with high bias
(see the left of Figure 3):

- Low data size causes training errors to be low and cross-validation errors to be high.
- Large data size causes both training and cross-validation errors to be high (very similar) and become stable.

This means that the algorithm suffers from high bias. So getting more training data will not be very useful for further improvement.

In case of a hypothesis with high variance
(see the right of Figure 3):

- Low data size causes training errors to be low and cross-validation errors to be high.
- Large data size causes training errors to increase and cross-validation errors to decrease (without stabilising).

This means that the algorithm suffers from high variance, so getting more training data may be useful.

Error metrics for skewed classes: The data sets is largely skewed when we have many more samples of one class compared to other classes in the entire data sets. In such a situation, we can use two attributes of the algorithm, called *Precision* and *Recall*, to evaluate it. In order to determine these attributes, we need to use the following truth table.

	PREDICTED	ACTUAL
TRUE POSITIVE	1	1
TRUE NEGATIVE	0	0
FALSE NEGATIVE	0	1
FALSE POSITIVE	1	0

So,

$$\text{Precision (P)} = \frac{\text{True Positives}}{\text{Total Number of Predicted Positives}} = \frac{\text{True Positives}}{\text{True Positives + False Positives}}$$

and

$$\text{Recall (R)} = \frac{\text{True Positives}}{\text{Total Number of Actual Positives}} = \frac{\text{True Positives}}{\text{True Positives + False Negatives}}$$

We need both *Precision* and *Recall* to be high for the algorithm to perform better. Instead of using these separately, there is a simpler solution—combining them in a single value called F1 score. The formula to calculate the F1 score is:

$$\text{F1 Score} = 2 (PR/P+R)$$

We need a high F1 score (so both P and R should be large). Also, the F1 score should be obtained over the cross-validation data set.

A few rules for running diagnostics are:

- More training examples fix high variance but not high bias.
- Fewer features fix high variance but not high bias.
- Additional features fix high bias but not high variance.
- The addition of polynomial and interaction features fixes high bias but not high variance.
- When using gradient descent, decreasing lambda can fix high bias and increasing lambda can fix high variance (lambda is the regularisation parameter).
- When using neural networks, small neural networks are more prone to underfitting and big neural networks are prone to overfitting. Cross-validation of the network size is a way to choose alternatives.

Tips on using the most common algorithms

SVM: SVM is one of the most powerful supervised algorithms. Listed below are some tips to applying the SVM algorithm:

- If the feature set (n) is large relative to the data sets size (m), then use SVM with logistic regression or linear kernel. We don't have enough examples, which would have required a complicated polynomial hypothesis.
- If n is small and m is intermediate, then use SVM with a Gaussian Kernel. We have enough examples so we may need a complex non-linear hypothesis.
- If n is small and m is large, then manually create/add more features, and use logistic regression or SVM without a kernel. Increase the features so that logistic regression becomes applicable

K-Means: K-Means is most popular and a widely used clustering algorithm for unsupervised data sets. Some tips on applying K-Mean are listed below:

- Make sure the number of your clusters is less than the number of your training examples.
- Randomly pick K training examples (be sure the selected examples are unique). Determine K by plotting it against the cost function. The cost function should reduce as we increase the number of clusters, and then flatten out. Choose K at the point where the cost function starts to flatten out.

The availability of so many machine learning libraries today makes it possible to start building self-learning applications. However, when we talk about applying machine learning algorithms to real business problems, they need to be highly generalised, scalable and accurate in their outcome – be it prediction or clustering. This is not a very easy task, and requires a process involving various steps to make the algorithms work in the desired fashion. END

References

This article is largely inspired by a machine learning course by Andrew Ng on Coursera.

By: Amit Badheka

The author is currently working as a senior technical architect with Capgemini India Pvt Ltd. He is a TOGAF certified enterprise architect with 17 years of IT experience in various capabilities. His areas of interest include various emerging technologies such as IoT, predictive analytics and cognitive computing. He can be reached at *amit.badheka@capgemini.com*.